

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

UC LINA 600 — Linear Algebra for Graphics

The graphics pipeline is linear algebra, so this is not background — it is the subject

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Prerequisite · take before Semester 1

Prerequisites: Secondary school algebra and trigonometry. UC MATH 600 is useful and is not required; the two can be taken in parallel

Effort: 8–10 hours per week · 13 modules + 2 project weeks

Deliverable: A small software renderer that draws a shaded, perspective-projected mesh — written with no graphics API and no matrix library, so that every transform in it is one you implemented

Course description

This is the second mathematics prerequisite, and it is less optional than it looks. The graphics pipeline is linear algebra — a vertex becomes a pixel by being multiplied by four matrices — and CSCE 641 begins using bases, projections, and change of basis in week two without defining any of them.

The first third builds the objects. Vectors, spans, bases, and matrices as linear maps (Modules 02 and 03) — and **Module 04 on change of basis, which is the single most useful idea in the course** and the one that makes the pipeline comprehensible rather than memorised.

The second third is the geometry. Dot products, projection, cross products, normals, and orientation (Modules 05 and 06) — **which is the vocabulary of every shading calculation** — and then **homogeneous coordinates and the pipeline itself** (Modules 07 and 08), where the course pays off visibly.

The third is the parts that matter later. Determinants and degeneracy (Module 09) explain why transforms fail; **eigenvectors** (Module 10) are needed for CSCE 649 and CSCE 753; **quaternions** (Module 11) are how every engine actually stores a rotation; and **least squares** (Module 12) is the fitting procedure underneath half of CSCE 633.

And the closing module is about verification, which is the program's rule in this subject. **A transform that looks right in one test scene is not a correct transform** — and the specific ways a wrong matrix looks right are worth knowing before you spend a weekend on one.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 Explain why linear algebra is the graphics prerequisite.
- 2 Work with vectors, spans, linear independence, and bases.
- 3 Read a matrix as a linear map, and compose maps.
- 4 Change basis, and explain why it is the central idea.
- 5 Use dot products, projection, and orthogonality.
- 6 Use cross products, normals, and orientation correctly.
- 7 Explain homogeneous coordinates and affine transforms.
- 8 Derive the model-view-projection chain.
- 9 Use determinants to detect degeneracy and handedness.
- 10 Compute and interpret eigenvectors.
- 11 Use quaternions for rotation, and say why.
- 12 Solve least squares problems and explain the normal equations.
- 13 Verify a transform rather than eyeballing it.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	Why This Is the Graphics Prerequisite	What actually happens to a vertex?
2	Vectors, Spans, and Bases	What do the numbers in a vector mean?
3	Matrices as Linear Maps	Why is matrix multiplication defined that way?
4	Change of Basis	The same arrow, described from two places. How do you convert?
5	Dot Products and Projection	Why does a dot product tell you about an angle?
6	Cross Products and Normals	Which way does the normal point?
7	Homogeneous Coordinates	Why does a 3D point have four coordinates?
8	The Pipeline as a Matrix Chain	Can you derive the projection matrix?
9	Determinants and Degeneracy	What does the determinant measure?
10	Eigenvalues and Eigenvectors	Which vectors come out pointing the same way they went in?
11	Quaternions	Why not just use three angles?

Wk	Module	Core question it answers
12	Least Squares	More equations than unknowns. Now what?
13	Claiming a Transform Is Correct	It looks right. Is it right?

Projects

Project 1 — Transforms by hand (after Module 6)

Implement the operations, and verify them against hand calculations.

Requirements

- **Vector and matrix operations implemented from scratch** — add, scale, multiply, transpose — **with no library**.
- **Dot and cross products**, with **the geometric meaning verified**: angle from the dot, perpendicularity and area from the cross.
- **Rotation, scale, and shear matrices** built and **composed in both orders**, with the difference shown and explained.
- **A change of basis performed and inverted** (Module 04), with the round trip verified to numerical precision.
- **Every result checked against a hand calculation** on a case you worked on paper first.
- **And each transform drawn**, so that the picture and the numbers agree.

Done means

- **The hand calculations submitted alongside the code** — **which is the point**, because code that agrees with itself proves nothing.
- **The composition order difference demonstrated and explained**, not just observed — **matrix multiplication is not commutative and this is where that becomes real**.
- **The change of basis round trip exact** to floating-point tolerance, with the tolerance stated.
- **And the drawings included**, because **a transform you cannot picture is one you will misuse**.

Project 2 — A software renderer (after Module 12)

Draw a shaded perspective mesh with every matrix your own.

Requirements

- **A full model-view-projection chain** (Module 08), **each matrix derived rather than copied**, with the derivation submitted.
- **A triangle mesh loaded and transformed**, with **per-face normals computed by cross product** (Module 06).
- **Diffuse shading from a dot product** (Module 05), and **backface culling from the sign of a determinant or a dot** (Module 09).
- **Perspective divide and a viewport transform**, with **the near plane's behaviour explained**.
- **A camera that orbits**, using either a rotation matrix or a quaternion (Module 11) — and **say which and why**.

- **And the verification suite** of Module 13 §2: known points, known answers, checked automatically.

Done means

- **Every matrix derived rather than copied**, with the derivations submitted — **a correct projection matrix found online teaches nothing**, and this project is the one place in the program where that matters most.
- **The verification suite passing**, and **at least one bug found by it rather than by looking at the image** (Module 13 §1).
- **The handedness convention stated** and consistent throughout — which is the failure this project exists to inoculate against.
- **And the near-plane behaviour explained**, because **understanding why geometry explodes near it is understanding the projection matrix** (Module 08 §3).

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
Strang — Introduction to Linear Algebra, with MIT 18.06 (OCW, full video)	The primary course, free in full. The lectures are the standard introduction for good reason, and the column-space framing is exactly the one this course uses.
3Blue1Brown — Essence of Linear Algebra (free video)	Watch this first, all of it, before anything else. Fifteen short videos that build the geometric intuition the algebra then formalises — and Modules 03, 04, and 09 are much easier afterwards.
Immersive Math — Interactive Linear Algebra (free)	Modules 02 through 06, free. Every figure is manipulable, which matters more here than in most subjects.

Resource	Role
Lengyel — Mathematics for 3D Game Programming and Computer Graphics	Modules 06 through 11. The graphics-specific treatment, including quaternions done properly. Library copy.
Gortler — Foundations of 3D Computer Graphics	Modules 04, 07, and 08. Unusually careful about frames and change of basis, which is exactly where graphics texts are usually sloppy. Library copy.
Shirley — Ray Tracing in One Weekend (free)	Project 2's companion. Free, short, and it exercises Modules 05, 06, and 09 on a real renderer.

Tooling

- **Python with numpy for learning, and nothing but raw arrays for Project 2 — because a matrix library will do the one thing you were supposed to understand.**
- **A plotting library — draw every vector and every basis you work with, since this is a subject where the picture is the understanding and the algebra is the bookkeeping.**
- **C++ for Project 2 if you intend to continue into CSCE 641, which will want it — and the renderer transfers directly.**
- **A right-handed convention, chosen once and written down — because handedness bugs are the single most common failure in this subject and they are invisible until a normal points the wrong way (Module 06 §3).**
- **And a test scene with known answers — a unit cube at the origin, a camera on an axis — so that every transform can be checked against a number you computed by hand (Module 13 §2).**

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.